Private Key Fragments Secure Recovery Approach in Passkeys System Based on Blockchain Technology and Error-Correcting Codes

1st ILBOUDO Assane
dept. of Computer Science
University Joseph KI-ZERBO
Ouagadougou, Burkina Faso
assaneilboudo1998@gmail.com

2nd BASSOLE Didier
dept. of Computer Science
University Joseph KI-ZERBO
Ouagadougou, Burkina Faso
dbassole@gmail.com

3rd GUEL Désiré
dept. of Computer Science
University Joseph KI-ZERBO
Ouagadougou, Burkina Faso
guel.desire@gmail.com

***Abstract*—In this paper, we propose an innovative approach to enhance the security and resilience of passkeys. this approach combines Shamir's Secret Sharing Scheme, Reed–Solomon error-correcting codes, AES-GCM encryption, and decentralized storage systems such as blockchain and IPFS. In this approach, the adopted methodology is structured around three main phases: (i) passkey fragmentation, (ii) distributed and secure storage of fragments, and (iii) secret reconstruction when required. We evaluated our scheme, referred to as *F2S-ECC* (Fast Fourier Shamir with Error-Correcting Codes), across multiple dimensions. First, the complexity analysis shows a performance of order $\mathcal{O}(n \log n)$, ensuring computational efficiency suitable for distributed environments. Second, the scalability evaluation of the decentralized storage mechanism highlights its improved ability to adapt to increasing data volume and the number of participants. Finally, the security of the scheme was validated through formal verification in the Dolev–Yao model, demonstrating that no attacks could compromise the confidentiality, integrity, or availability of passkeys. These results emphasize the relevance of our approach in addressing the challenges of secure protection and recovery of passkeys in distributed and low-trust environments.**

***Index Terms*—Passkey System, Blockchain, Error-Correcting Codes, Security**

I. Introduction

Based on asymmetric cryptography, Passkeys constitute a passwordless authentication solution which is generating increasing interest due to its ability to offer a more robust and secure alternative to traditional passwords. By eliminating the reliance on memorized secrets and centralized password databases, passkeys enhance security and improve the user experience. However, despite their undeniable advantages, managing passkeys raises new challenges, particularly in terms of backup and recovery. Indeed, the loss or failure of the device where passkeys are stored can result in irreversible loss of access, due to the lack of reliable preservation and restoration mechanisms. The main objective of this work is to propose an innovative approach to ensure the resilience and security of passkey storage, even in the event of device loss. Our contribution combines several complementary techniques: Shamir's secret sharing scheme, to split the key into secure fragments; Reed–Solomon error-correcting codes, to enhance error tolerance; AES-GCM symmetric encryption, to protect the confidentiality of the fragments; and the use of decentralized systems, such as blockchain and IPFS, to ensure data availability and integrity. This approach thus aims to provide a comprehensive and robust solution to the challenge of secure passkey backup in a distributed and potentially hostile environment. The remainder of this paper is organized as follows: Section II reviews the related work. Section III presents our methodological approach. Section IV evaluates the computational complexity of the proposed scheme, while Section V discusses its scalability. Section VI provides the formal security analysis. Finally, Section VII concludes the paper.

II. Related Work

The protection of sensitive data has motivated extensive research on secret sharing as a core cryptographic technique. Classical schemes, such as Shamir's scheme with reconstruction complexity $\mathcal{O}(k^2)$ and Blakley's scheme with $\mathcal{O}(k^3)$ [1], [2], provide information-theoretic security but lack mechanisms for detecting or correcting corrupted shares. To address these limitations, Patel et al. (2024) introduced RB-SS [3], which combines linear erasure codes and randomly banded matrices. RB-SS achieves improved efficiency and resilience with reconstruction complexity $\mathcal{O}(n(\lambda + \log n))$, though it still lacks error-correction capabilities. Similarly, Aureliano et al. (2025) proposed PETS [4], which minimizes share size using pseudorandom encryption under a computational security model, but it also does not support error detection or correction. On the efficiency side, Voudouris et al. (2025) [5] evaluated interpolation methods in Shamir's scheme, demonstrating that FFT-based reconstruction reduces complexity to $\mathcal{O}(n \log n)$ compared to $\mathcal{O}(n^2)$ for classical approaches. However, even this optimized method does not address the issue of erroneous shares. Finally, Beydili et al. (2025) [6] applied Shamir's scheme to blockchain-based e-voting, leveraging Ethereum to ensure anonymity and privacy.

Despite its benefits, this approach faces $\mathcal{O}(n^2)$ complexity and scalability challenges for large-scale deployments.

The analysis of existing secret sharing schemes highlights a trade-off between computational efficiency, fragment compactness, and error resilience. Classical approaches, such as those of Shamir and Blakley, provide strong information-theoretic security but remain vulnerable to fragment loss or corruption. More recent contributions, such as RB-SS and PETS, have improved performance and reduced fragment size, yet they still lack error-correction mechanisms. Furthermore, the approach of storing data directly on-chain faces significant scalability limitations. To overcome these challenges, we propose F2S-ECC (Fast Fourier Shamir with Error-Correcting Codes), a scheme that integrates error-correcting codes with a hybrid on-chain/off-chain storage model, thereby offering a robust, efficient, and scalable solution.

III. Methodological approach

In this section, we present a solution designed to address the issues identified in the literature review. The proposed approach is structured into two (02) complementary blocks (a, b), presented in Figure 1.

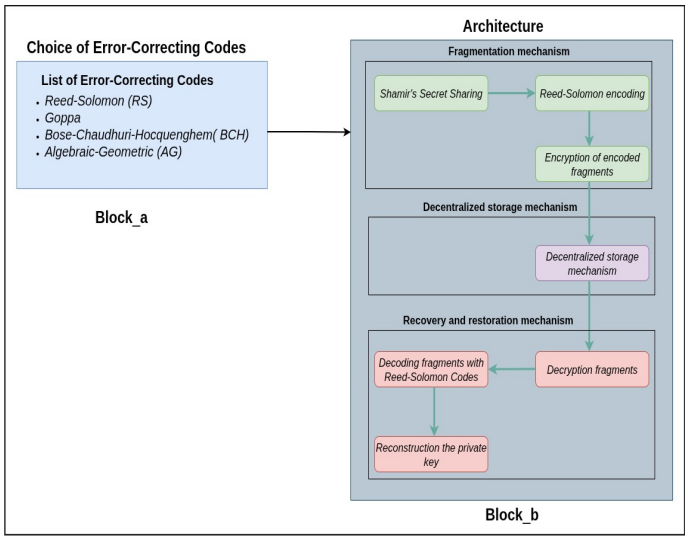

Fig. 1. Methodological process

A. Choice of Error-Correcting Codes

The choice of Error-Correcting Codes is based on a set of key criteria (Field,FFT compatibility,Distance, Complexity) to ensure the robustness and efficiency of the scheme

TABLE I
Comparison of main families of error-correcting codes

Code	Field	FFT Compatible	Distance (d)	Complexity
RS [2], [7]	$GF(q)$	Yes	$n-k+1$	$O(n^2)$
Goppa [7]	$GF(2^m)$	No	$\overset{>}{\frac{n-k}{\log n}}+1$	$O(n^3)$
BCH [7]	$GF(2)$	No	$2t+1$	$O(t^2)$ or $O(n\log n)$
AG [8]–[10]	$GF(q^2)$	Partial	$n-k+1-g$	$O(n^3)$

Reed–Solomon (RS) codes, thanks to FFT optimizations, achieve a decoding complexity of $O(n\log n)$ compared to $O(n^2)$ for traditional approaches, making them the reference solution for Shamir's Secret Sharing (SSS) schemes requiring high performance. In comparison, BCH codes (over $GF(2)$ or $GF(2^m)$) remain less efficient, while Goppa codes, although resistant to quantum attacks, are limited by their high computational cost of $O(n^3)$. RS codes stand out for their optimal minimum distance, native FFT compatibility, and flexibility in choosing (n,k) parameters. In high-threshold configurations, they guarantee confidentiality as long as the threshold t is not reached. To combine information-theoretic security, error resilience, and computational efficiency, a two-layer cryptographic architecture is proposed: (1) applying Reed–Solomon coding to SSS fragments to enhance error tolerance; (2) encrypting these fragments with AES-GCM, thereby ensuring both confidentiality and authenticity.

B. Architecture

we present an innovative architecture in Figure 2 for secure backup and recovery of a private key in a decentralized environment. This architecture relies on the synergistic integration of three main components, presented in Figure(Block_b) 1 :

- a fragmentation mechanism combining Secret Sharing Scheme (SSS), Reed–Solomon Error-Correcting Code, and symmetric encryption;
- a decentralized storage mechanism based on blockchain and IPFS network;
- a recovery and restoration mechanism based on symmetric decryption, Reed–Solomon and reconstruction of the private key according to Shamir's scheme.

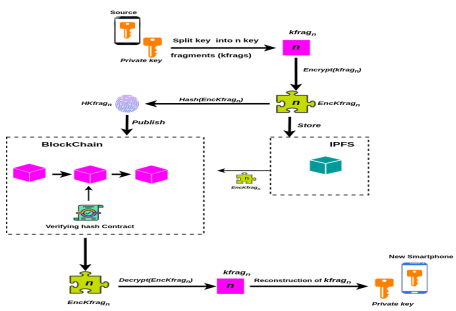

Fig. 2. Storing and reconstructing key fragments mechanism

The objective of this architecture is to guarantee the confidentiality, integrity, and availability of the private key, while enabling its secure restoration on a new device. Concretely, the private key is first fragmented into several shares using the secret sharing scheme and encoded with the Reed–Solomon code, then each fragment is individually encrypted before being stored in a distributed environment. Blockchain ensures verifiability and integrity of the fragments by publishing their hashes and executing a validation smart contract. During the restoration phase, the encrypted fragments are retrieved from the distributed system, their integrity is verified against the hashes recorded on the blockchain, then each fragment is decrypted and corrected if necessary using the Reed–Solomon code. Finally, the valid fragments are combined according to Shamir's scheme to reconstruct the original private key on the new device. This approach thus provides a robust solution against errors, malicious attacks, and potential failures, while maintaining high performance and security levels for private key management and recovery.

1) Fragmentation mechanism: In this section, we detail the process of our backup mechanism, which consists of three main phases: applying a secret sharing scheme to fragment the private key into several distinct shares; encoding the fragments resulting from Shamir's scheme using a Reed–Solomon code, to add redundancy and increase error resilience; individually encrypting the fragments before storing them in the decentralized environment.

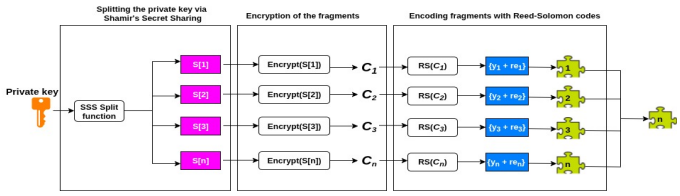

Fig. 3. Fragmentation process

This process ensures confidentiality, fault tolerance, and data integrity throughout the backup cycle.

Phase 1: Shamir's Secret Sharing: The first step consists of dividing the private key K into n distinct fragments, according to a (k, n) threshold scheme proposed by Adi Shamir. This mechanism is based on the fundamental properties of polynomials over a finite field and guarantees theoretical information-theoretic security of the secret as long as fewer than k fragments are collected.

a) Secret polynomial construction.: We construct a polynomial $f(x)$ of degree $k-1$ over a finite field $\mathbb{F}_q$, as:

$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{k-1} x^{k-1}$ where: the constant term $a_0 = K$ corresponds to the private key to protect; the coefficients $a_1, a_2, \ldots, a_{k-1}$ are chosen randomly and independently in $\mathbb{F}_q$, ensuring uniqueness and unpredictability of the fragments.

b) Fragment generation: Then, n nonzero and distinct abscissas x_i $(i = 1, 2, \ldots, n)$ are selected in the finite field. For each x_i, the polynomial $f(x)$ is evaluated to produce the pair: $S[i] = \big(x_i, f(x_i) \pmod q\big)$

Algorithm 1 Fragmentation mechanism

Require: Private key K, threshold (k, n)
Ensure: Encrypted fragments $\{C_i\}$

▷ Secret polynomial construction

$f(x) \leftarrow K + a_1 x + \cdots + a_{k-1} x^{k-1}$ over $\mathbb{F}_q$, with random a_j

▷ SSS fragments generation

for $i = 1$ **to** n **do**
  Select distinct x_i
  $S[i] \leftarrow (x_i, f(x_i))$
end for

▷ AES-GCM encryption an hash function

for $i = 1$ **to** n **do**
  $C_i \leftarrow$ AES-GCM$(S[i])$
  $H_i \leftarrow$ sha256(C_i)
end for

▷ Reed–Solomon encoding

for $i = 1$ **to** n **do**
  Encode C_i with RS to obtain $y_i + r_i$
end for
return $\{y_i + r_i, H_i\}$

Each pair $S[i]$ thus constitutes an independent fragment of the original secret.

Phase 2: Encryption of encoded fragments: The encryption layer constitutes the third and final protection barrier in the proposed hybrid scheme. Its primary objective is to ensure the *confidentiality* and *integrity* of the fragments generated during the previous phase, namely secret sharing (SSS), particularly when they are stored or transferred in distributed environments often exposed to external threats. Each fragment $S[i]$, produced at the end of Shamir's Secret Sharing phase, undergoes authenticated encryption. To this end, the scheme uses the *AES (Advanced Encryption Standard)* algorithm in *Galois/Counter Mode (AES-GCM)*, which combines the following properties: **Confidentiality**, by masking the content of the fragments; **Authenticity** and **integrity verification**, through the generation of an authentication tag attached to the encrypted data. Formally, the encrypted and hashed fragment C_i are obtained according to the following relation: $C_i =$ Encrypt$(S[i])$ and $H_i =$ Hash(C_i) where C_i=$EncKfrag_n$, H_i=$Hkfrag_n$ denote the encrypted, hashed and authenticated version of the fragment $S[i]$. This step thus completes the *defense-in-depth* mechanism of the hybrid scheme, protecting the fragments against any form of unauthorized access or malicious tampering, while preserving their reliability and confidentiality.

Phase 3: Reed–Solomon encoding: In the second phase, each fragment C_i is protected against potential transmission or storage errors by being encoded using an error-correcting code from the Reed–Solomon (RS) family. This encoding strengthens the robustness of the system by enabling the detection and correction of errors during the secret reconstruction phase.

The process is structured as follows:

c) Input Block: Each encrypted fragment C_i is first represented as a sequence of symbols defined over an appropriate finite field, typically $\mathbb{F}_{2^m}$. This representation ensures compatibility with the RS encoding scheme, which operates on symbols from a finite alphabet rather than on individual bits.

d) Encoding Process: The Reed–Solomon encoder then transforms the input sequence into an extended codeword, of the form: $RS(C_i) = (y_n + re_n)$ where: $(y_1, y_2, \ldots, y_n)$ denote the information symbols, directly derived from the original encrypted fragment C_i; $(re_1, re_2, \ldots, re_n)$ correspond to the redundancy symbols generated by the RS encoder. These redundancy symbols enable the detection and correction of up to a certain number of errors or erasures in the codeword during the decoding phase. The exact number of correctable errors depends on the RS code parameters, in particular its length and minimum distance.

e) Objective: RS encoding enhances the resilience of secret sharing against errors or partial losses. As long as these remain within the code's correction capacity, the integrity of the secret is guaranteed during reconstruction.

2) Decentralized storage mechanism: In this section, we present the process of storing fragments in a decentralized system in Figure 4. First, a cryptographic hash of each encrypted fragment C_i, denoted $Hkfrag_n$, is computed in phase 2. This hash plays a fundamental role as an integrity proof, allowing the detection of any malicious or accidental modification of the fragments. Specifically, the hash $Hkfrag_n$ is published on the Blockchain through a smart contract (*verifying hash contract*). The Blockchain, as an immutable and distributed ledger, guarantees the integrity and non-repudiation of the fragments. In parallel, each encoded fragment by RS encoder $y_n + re_n$ is stored in IPFS, a distributed and decentralized file system, which ensures data persistence and availability while avoiding single points of failure. Storing on IPFS produces a unique content identifier (*CID*) for each fragment, which depends on the exact stored content. This CID is also recorded on the Blockchain, alongside the hash $Hkfrag_n$. Thus, the Blockchain serves a dual purpose: it provides the *CIDs* required to retrieve the fragments from IPFS, and it enables verification of their integrity by comparing the recomputed hash after retrieval with the one stored.

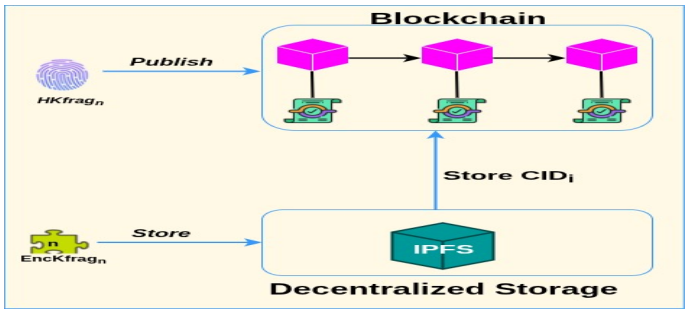

Fig. 4. Data storage procedure

This approach ensures not only the cryptographic security of the fragments but also their traceability and availability in a distributed environment.

Algorithm 2 Decentralized storage mechanism

Require: Encoded fragments by RS encoder $\{y_n + re_n\}$
Ensure: Records (CID_i, H_i) on the blockchain
  for $i = 1$ **to** n **do**
    $H_i \leftarrow \text{Hash}(C_i)$
    $CID_i \leftarrow \text{IPFS.store}(y_n + re_n)$
    Publish (CID_i, H_i) via smart contract
  end for
  return $\{(CID_i, H_i)\}$

3) Recovery and restoration mechanism: In this section, we rigorously present the private key reconstruction process in Figure 5. First, the encoded fragments by RS encoder frag$_n$ are retrieved from the distributed storage environment IPFS, thus ensuring their accessibility in a potentially decentralized system. The integrity of the fragments Hk$_{C_i}$ is then verified using cryptographic hashes previously recorded on the blockchain, ensuring that the data has not been altered by a malicious third party or transmission errors. the Reed–Solomon error-correcting code is applied to decode the $y_n + r_e$ and correct possible corruptions or data losses that occurred during the previous operations. Subsequently,the fragments C_i are then decrypted using the symmetric AES-GCM algorithm, allowing restoration of their original content while preserving the confidentiality of the information during the storage phase. At the end of these steps, the valid fragments $S[i]$, obtained after decryption and decoding, are used within Shamir's Secret Sharing (SSS) scheme.

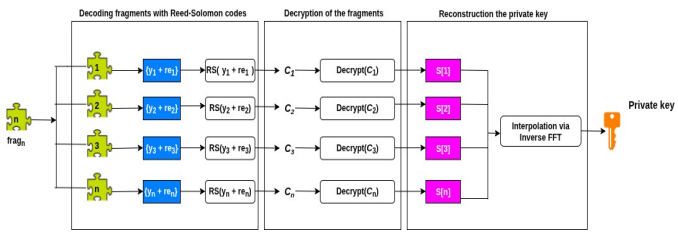

Fig. 5. Recovery and restoration process

The reconstruction of the private key relies on an efficient and optimized polynomial interpolation using the Fast Fourier Transform (FFT). Classical interpolation methods, such as Lagrange, Newton, or barycentric, exhibit a quadratic complexity $O(n^2)$ when applied from k valid fragments. In contrast, the use of FFT reduces this complexity to $O(n \log n)$, provided that the evaluation points x_i are chosen as n-th roots of unity in $\mathbb{F}_q$. The first step is to extend the vector of k values $[y_0, y_1, \ldots, y_{k-1}]$, which correspond to the known polynomial evaluations at k distinct abscissae, by appending $n-k$ zeros to obtain a vector of length $n \geq k$. This extension allows leveraging the properties of the FFT on a standard size n. Then, the inverse fast Fourier transform is applied to this extended vector. Formally, the inverse fast Fourier transform on n points

is given by: $a_m = \frac{1}{n}\sum_{j=0}^{n-1} y_j \cdot \omega^{-jm}, \quad m = 0, \ldots, n-1$, where: y_j denotes the j-th polynomial evaluation; $\omega = e^{2\pi i/n}$ is a primitive n-th root of unity; and ω^{-1} is its inverse. This transformation directly provides the coefficients of the polynomial $P(x)$ in the form $[a_0, a_1, \ldots, a_{n-1}]$, where each a_j corresponds to the coefficient of the degree j term. Finally, the original secret S is recovered by extracting the constant term a_0, since by definition $P(0) = a_0$.

Algorithm 3 Recovery and restoration mechanism

Require: Records (CID_i, H_i) on the blockchain
Ensure: Restored private key K
  for $i = 1$ **to** n **do**
    $C_i \leftarrow$ IPFS.get(CID_i)
    $H'_i \leftarrow$ Hash(C_i)
    if $H'_i \neq H_i$ **then**
      Mark C_i as corrupted
    end if
  end for
  Decode and correct errors $\rightarrow C_i$
  for all valid C_i **do**
    $S[i] \leftarrow$ AES-GCM$^{-1}(C_i)$
  end for
  Interpolate $f(x)$ and extract $K = f(0)$
  return K

IV. Computational Complexity

The experiments evaluate the F2S-ECC scheme by varying several parameters: a 256-bit private key (key size),threshold configurations (k, n) ranging from $(2, 3)$ to $(20, 32)$, and a Reed–Solomon code over $\mathrm{GF}(2^8)$ with redundancy rates $r = 16$, 32, or 64. The objective was to analyze both the computational complexity and the error resilience of the scheme, in comparison with existing approaches reported in the literature. The integration of the Fast Fourier Transform (FFT) into the polynomial evaluation and interpolation processes allows for a significant reduction in the complexity of Shamir's Secret Sharing as well as Reed–Solomon codes, which are traditionally employed in secret-sharing mechanisms and error-correcting schemes. The experimental results, presented in Figure 6, highlight a markedly slower growth in computation time for our scheme compared to conventional methods. This trend confirms F2S-ECC's ability to maintain efficient scalability even in large-scale systems. Moreover, the F2S-ECC scheme, as well as the approaches proposed by Patel et al. [3] and Voudouris et al. [5], exhibit enhanced robustness against errors compared to other existing methods reported in the literature. The Figure 6 shows that the approaches of Patel et al. [3] and Voudouris et al. [5], both characterized by a complexity of $O(n \log n)$, are at the same level as the F2S-ECC scheme. Consequently, the computational complexity of the F2S-ECC scheme is also evaluated as $O(n \log n)$.

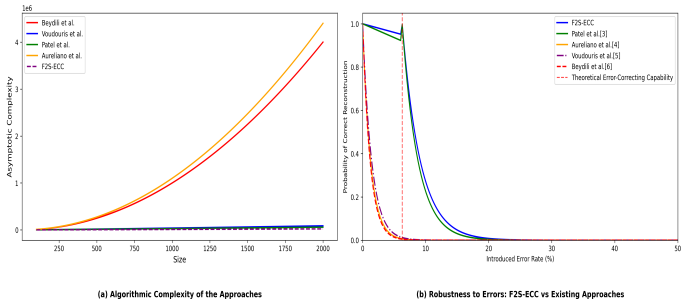

Fig. 6. Computational Complexity versus Error Resilience

Furthermore, the comparative analysis reveals that F2S-ECC provides a better trade-off between computational efficiency and error tolerance, outperforming state-of-the-art approaches. These findings underscore the relevance of jointly integrating FFT and error-correcting codes within the secret-sharing framework, and emphasize the potential of our scheme for applications requiring both robustness and high performance at large scale.

V. Scalability

To assess the scalability of our proposed hybrid on-chain/off-chain mechanism, we conducted a series of experiments comparing the performance of the Ethereum blockchain used alone (pure on-chain storage) with that of Ethereum coupled with IPFS (off-chain storage). The experiments were conducted on a hardware platform equipped with an Intel Core i7 processor clocked at 2.59 GHz and 32 GB of RAM, running Ubuntu 24.04 LTS. The software environment included Ganache CLI for blockchain simulation, Python with the Web3.py library for automation and interaction with smart contracts, Solidity for smart contract development, and IPFS for decentralized storage of experimental data. The evaluation relied on four main indicators: transaction cost, gas consumption, throughput, and latency. As illustrated in Figures 7 and 8, direct data storage on Ethereum leads to a significant increase in transaction costs and gas consumption, particularly as file size grows. In contrast, the Ethereum–IPFS hybrid approach incurs a relatively constant overhead, since only cryptographic hashes are recorded on-chain, while large data volumes are externalized and stored off-chain. This model thus ensures greater cost efficiency, especially in scenarios involving large-scale data storage.

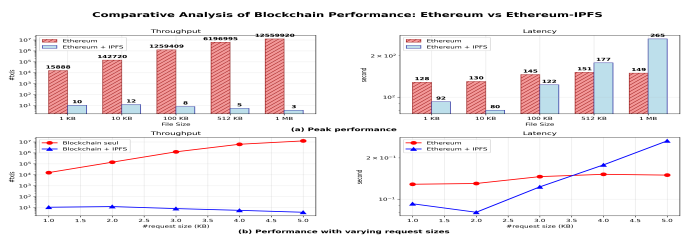

Fig. 7. Analysis of Throughput and Latency — Comparison between Ethereum and Ethereum–IPFS

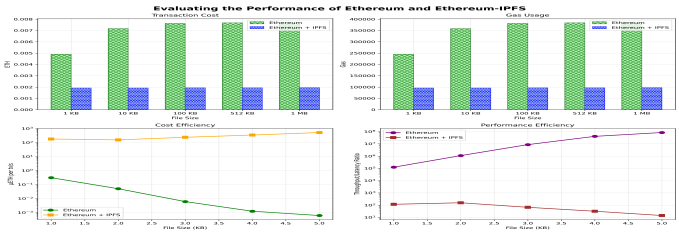

Fig. 8. Evaluation of Gas Costs and Consumption — Comparison between Ethereum and Ethereum–IPFS

From a performance perspective, Ethereum alone achieves very high throughput rates for small to medium-sized files, but at the expense of increased latency when data sizes approach the megabyte scale. Conversely, the Ethereum + IPFS integration exhibits lower throughput due to the additional overhead of distributed data retrieval, while maintaining lower latency for small to intermediate file sizes. These results highlight a trade-off between raw throughput performance (favored by Ethereum alone) and improved scalability through reduced on-chain congestion, made possible by the hybrid model. Overall, the findings demonstrate that the hybrid on-chain/off-chain mechanism significantly enhances scalability by lowering transaction costs and gas consumption while maintaining acceptable performance in terms of throughput and latency. By externalizing large data volumes to IPFS, the system effectively mitigates the inherent scalability limitations of Ethereum, making it a promising approach for real-world applications that require both data integrity and economic efficiency.

VI. Formal Security Analysis

A. Dolev–Yao Formal Model

The Dolev–Yao formal model is one of the most widely adopted theoretical frameworks for the analysis of cryptographic protocols [11]–[13]. It defines an abstract threat model that enables reasoning about security properties without delving into the mathematical intricacies of cryptography. In this model, the adversary is considered active and has full control over the communication network illustrated in Figure 9. This implies that the attacker can: Intercept communications between legitimate entities. Modify or inject new messages in order to manipulate the protocol. Replay previously observed messages.

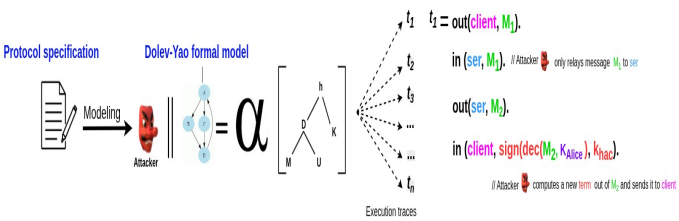

Fig. 9. Execution and Analysis of Attack Scenarios in the Formal Dolev–Yao Framework

Furthermore, the adversary is assumed to be proficient in the available cryptographic primitives. However, cryptography itself is treated as a black box: the attacker cannot directly break the underlying cryptographic mechanisms (e.g., invert a hash function or guess a secret key). Instead, the adversary's interaction with cryptography is limited to the use of the functionalities provided, such as encryption, decryption with the appropriate key, hashing, concatenation, and similar operations. The Dolev–Yao model relies on representing messages as formal terms, built within a term algebra [14].

a) Set of function symbols: Messages are generated using symbols that represent cryptographic operations. For example: `enc(m, k)`: encryption of message m with key k; `dec(m, k)`: decryption of message m with key k; `pair(m1, m2)`: concatenation or pairing of two messages; `sign(m, k)`: signature of message m with private key k.

b) Structural equivalence: Equivalences are defined to capture the expected behavior of cryptographic primitives. For example: $dec(enc(m,k),k) = m$ and $verify(sign(m,k_{priv}),k_{pub}) = m$.
This modeling approach makes it possible to reason about possible message transformations and to determine whether an adversary can, from a given set of observed terms, reconstruct or forge a particular message illustrated in Figure 10.

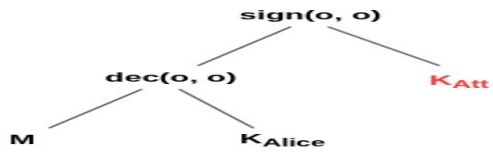

Fig. 10. Tree representation of a term in the Dolev–Yao model

B. Robustness Evaluation Using ProVerif Tool

The robustness of our approach was assessed using ProVerif, a reference tool for the formal analysis of cryptographic protocols based on the Dolev–Yao model. This analysis provides rigorous mathematical guarantees, which are essential for the security of critical systems (aeronautics, railway, nuclear, medical devices), where empirical methods such as simulations, penetration testing, or fuzzing remain partial [15]–[17]. We formally verified the following security properties: Confidentiality: private keys remain inaccessible to an adversary, even in the case of full control over communication channels; Authenticity and integrity of fragments: each fragment is strictly bound to its owner, and any unauthorized modification is detectable; Freshness and non-repudiation: each storage and retrieval operation is associated with a unique identifier, preventing replay attacks.

We analyzed our key fragment approach using ProVerif in order to verify the essential security properties: confidentiality, authenticity, integrity, provenance, and access control. The results obtained, illustrated in Figure 11 are detailed below:

a) Confidentiality of the secret: `Query not attacker(secret_key[]) is true.` The initial secret (`secret_key`) remains confidential. Even with full

control of the network, the adversary cannot derive or learn the value of the key.
Result: confidentiality guaranteed.

b) Link between reconstruction and initial storage: `Query event(secret_reconstructed(x_1)) ==> event(begin_storage(x_1)) is true.` Any reconstruction of the secret can only occur if the owner has previously initiated the storage process.
Result: authenticity of reconstruction guaranteed.

c) Reconstruction conditioned by integrity: `Query event(secret_reconstructed( secret_key[])) ==> event( integrity_verified(f,h_1)) is true.` The reconstruction of the secret depends on the verification of fragment integrity, preventing recovery from corrupted data.
Result: integrity of fragments guaranteed.

d) Retrieval only after storage: `Query event(fragment_retrieved(f,id_1)) ==> event(fragment_stored_ipfs(f,id_1)) is true.` Any retrieved fragment necessarily originates from legitimate storage on IPFS, ensuring data reliability and provenance.
Result: provenance guaranteed.

e) Absence of unauthorized access: `Query not event( unauthorized_access_attempt(f)) is true.` No illegitimate access to fragments or sensitive data is possible, even under a powerful Dolev–Yao adversary model.
Result: protection against unauthorized access guaranteed.

```
Translating the process into Horn clauses...
Completing...
Starting query not event(unauthorized_access_attempt(f))
RESULT not event(unauthorized_access_attempt(f)) is true.
--------------------------------------------------------------
Verification summary:

Query not attacker(secret_key[]) is true.

Query event(secret_reconstructed(x_1)) ==> event(begin_storage(x_1)) is true.

Query event(secret_reconstructed(secret_key[])) ==> event(integrity_verified(f,h_1)) is true.

Query event(fragment_retrieved(f,id_1)) ==> event(fragment_stored_ipfs(f,id_1)) is true.

Query not event(unauthorized_access_attempt(f)) is true.
--------------------------------------------------------------
```

Fig. 11. Overview of ProVerif Simulation Results for Our Approach

The verified approach is robust and resilient: it guarantees the main security properties with no vulnerabilities detected in the Dolev–Yao model, thanks to fragment sharing, IPFS, and blockchain.

VII. Conclusion

In summary, we have presented an innovative mechanism for the storage and recovery of passkeys. This approach ensures confidentiality through individual encryption of the fragments, preserves integrity by verifying the hashes recorded on the blockchain, and guarantees availability and resilience through the use of distributed storage on IPFS. As a continuation of this work, we plan to conduct an in-depth evaluation of the robustness, performance, and security of the proposed mechanism for storing and reconstructing private key fragments. Our approach ensures a robust and reliable management of passkey storage and recovery. The results demonstrate that our scheme achieves a computational complexity of $\mathcal{O}(n \log n)$, provides improved scalability with respect to increasing data volume and participants, and effectively resists attacks within the formal Dolev–Yao model.

References

[1] Wijaya, A., Alamsyah, I. M., & Barra, A. (2024). Threshold Changeable Secret Sharing (TCSS) via Skew Polynomials. European Journal of Pure and Applied Mathematics, 17(3), 1751-1761.

[2] Cao, H., Lin, S. J., & Zhang, W. (2025). Trimmed FFTs and Applications to Coding and Secret Sharing. IEEE Transactions on Communications.

[3] Patel, S., Persiano, G., Seo, J. Y., & Yeo, K. (2024, December). Efficient secret sharing for large-scale applications. In Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (pp. 3065-3079).

[4] Aureliano, I. L., Cohen, A., & D'Oliveira, R. G. (2025). Optimal Computational Secret Sharing. arXiv preprint arXiv:2502.02774.

[5] Voudouris, A., Tressos, A., Zarras, A., & Xenakis, C. (2025). Game on: a performance comparison of interpolation techniques applied to Shamir's secret sharing. The Computer Journal, 68(3), 261-272.

[6] Beydili, E., Cabuk, U. C., Dalkilic, G., & Ozturk, Y. (2025, April). Securing Blockchain-Based E-Voting Through Shamir's Secret Sharing over Ethereum. In 2025 13th International Symposium on Digital Forensics and Security (ISDFS) (pp. 1-6). IEEE.

[7] Blahut, R. E. (2003). Algebraic Codes for Data Transmission

[8] Shum, K. W., Aleshnikov, I., Kumar, P. V., Stichtenoth, H., & Deolalikar, V. (2003). A low-complexity algorithm for the construction of algebraic-geometric codes better than the Gilbert-Varshamov bound. IEEE Transactions on Information Theory.

[9] Stichtenoth, H. (2009). Algebraic Function Fields and Codes (2nd ed.). Springer-Verlag. (Chapitre 8)

[10] Høholdt, T., van Lint, J. H., & Pellikaan, R. (1998). Algebraic geometry codes. In Handbook of Coding Theory.

[11] De Vincenzi, M., Bodei, C., Costa, G., & Matteucci, I. Poster: Umarell–A New Attacker in ITS Communications.

[12] Boureanu, I., Wesemeyer, S., Rajaona, F., Schneider, S., & Treharne, H. A Systematic Study of Practical & Formal Privacy in the 5G AKMA Procedure.

[13] Assane, I., Didier, B., Pegdwindé, K. J., Gouayon, K., & Oumarou, S. (2025, May). Assessing Robustness and Resistance to Attacks of an Authentication System Based on OpenID Connect Protocol and Ethereum Blockchain. In 2025 15th International Conference on Electrical Engineering (ICEENG) (pp. 1-6). IEEE.

[14] Ammann, M., Hirschi, L., & Kremer, S. (2023). DY Fuzzing: les modèles Dolev–Yao formels rencontrent les tests de fuzz des protocoles cryptographiques. Cryptology ePrint Archive, Paper 2023/057.

[15] Lattuada, A., Hance, T., Bosamiya, J., Brun, M., Cho, C., LeBlanc, H., ... & Parno, B. (2024, November). Verus: A practical foundation for systems verification. In Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (pp. 438-454).

[16] Curaba, C., Denis, D. A., & Minisini, A. CryptoFormalEval: Integrating Large Language Models and Formal Verification for Automated Cryptographic Protocol Vulnerability Detection. In The First Workshop on System-2 Reasoning at Scale, NeurIPS'24.

[17] Yan, M., Bourgeat, T., & Malik, S. (2024). Formal Verification for Secure Processors: A Guide for Computer Architects. Computer, 57(10), 138-143.